Rental Price Prediction – Kuala Lumpur & Selangor

Problem Statement

Rent in KL and Selangor depends on many things: location, size, rooms, furnishing and property type. It is hard for tenants and owners to know if a listed rent is fair.

Who is the user? People looking for a place to rent in KL or Selangor (tenants). Before they sign a contract, they want to know: *"Is this rent fair, or am I overpaying?"*

- **Objective:** build a machine learning model that predicts the fair monthly rent of a property from its features.
- **Why it matters:** tenants can avoid overpaying, negotiate with a real number, and find areas that fit their budget.
- **Expected outcome:** a trained regression model and a React web app called **Houses Rental** where a tenant can:
  1. predict the monthly rent of a house from its details, and check if a listing's asking rent is fair
  2. find which areas fit their budget for the unit they want
  3. see market insights (which factors make rent higher)

Dataset

- **Source:** Kaggle – Rent Pricing Kuala Lumpur, Malaysia (scraped from mudah.my) https://www.kaggle.com/datasets/ariewijaya/rent-pricing-kuala-lumpur-malaysia
- **Size:** 19,991 listings, 14 columns
- **Target variable:** `monthly_rent`
- **Features used:** location, property type, rooms, parking, bathroom, size, furnished, region

Step 1: Import the libraries

In [1]:
```
# Tools for tables and numbers
import pandas as pd
import numpy as np
import json                 # to save the model for the website

# Tools for charts
import matplotlib.pyplot as plt
import seaborn as sns

# Tools for machine learning (scikit-learn)
from sklearn.model_selection import train_test_split    # split data into train
from sklearn.preprocessing import StandardScaler        # scale the numbers
from sklearn.linear_model import LinearRegression, LogisticRegression
```

```
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import classification_report, ConfusionMatrixDisplay, RocCu

# Chart settings: white background with grid, and a normal size
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)

# Show all the columns when we print a table
pd.set_option("display.max_columns", None)

print("Libraries imported successfully")
```

```
Libraries imported successfully
```

Step 2: Load the dataset

In [2]:

```
# Read the CSV file into a table (called a DataFrame)
df = pd.read_csv("mudah-apartment-kl-selangor.csv")

# shape = (number of rows, number of columns)
print("Dataset shape (rows, columns):", df.shape)

# Show the first 5 rows
df.head()
```

```
Dataset shape (rows, columns): (19991, 14)
```

Out[2]:

	ads_id	prop_name	completion_year	monthly_rent	location	property_type	roo
0	100323185	The Hipster @ Taman Desa	2022.0	RM 4 200 per month	Kuala Lumpur - Taman Desa	Condominium	
1	100203973	Segar Courts	NaN	RM 2 300 per month	Kuala Lumpur - Cheras	Condominium	
2	100323128	Pangsapuri Teratak Muhibbah 2	NaN	RM 1 000 per month	Kuala Lumpur - Taman Desa	Apartment	
3	100191767	Sentul Point Suite Apartment	2020.0	RM 1 700 per month	Kuala Lumpur - Sentul	Apartment	
4	97022692	Arte Mont Kiara	NaN	RM 1 299 per month	Kuala Lumpur - Mont Kiara	Service Residence	

In [3]:
```
# Show each column, its data type, and how many values are not empty
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19991 entries, 0 to 19990
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   ads_id                19991 non-null  int64  
 1   prop_name             19043 non-null  object 
 2   completion_year       10806 non-null  float64
 3   monthly_rent          19989 non-null  object 
 4   location              19991 non-null  object 
 5   property_type         19991 non-null  object 
 6   rooms                 19985 non-null  object 
 7   parking               14289 non-null  float64
 8   bathroom              19985 non-null  float64
 9   size                  19991 non-null  object 
 10  furnished             19986 non-null  object 
 11  facilities            17782 non-null  object 
 12  additional_facilities  14043 non-null  object 
 13  region                19991 non-null  object 
dtypes: float64(3), int64(1), object(10)
memory usage: 2.1+ MB
```

Step 3: Data cleaning

3.1 Find the missing values

In [4]:
```
# Count the empty values in each column, biggest first
print("Missing values per column:")
df.isnull().sum().sort_values(ascending=False)
```

```
Missing values per column:
```

Out[4]:
```
completion_year          9185
additional_facilities    5948
parking                  5702
facilities               2209
prop_name                 948
rooms                       6
bathroom                    6
furnished                   5
monthly_rent                2
ads_id                      0
location                    0
property_type               0
size                        0
region                      0
dtype: int64
```

In [5]:
```
# ads_id is different for every ad, so we remove it first.
# Then two copies of the same listing look exactly the same.
df = df.drop(columns=["ads_id"])

print("Rows before:", len(df))
df = df.drop_duplicates()          # remove rows that are exactly the same
print("Rows after:", len(df))
```

```
Rows before: 19991
Rows after: 18450
```

3.3 Fix the data types

`monthly_rent` is text like `"RM 4 200 per month"` and `size` is text like `"1842 sq.ft."`. We remove the extra words and change them to numbers. `errors="coerce"` turns anything that is not a number (like `"More than 10"`) into empty (NaN).

In [6]:
```
# monthly_rent: "RM 4 200 per month" -> "4200" -> 4200
df["monthly_rent"] = df["monthly_rent"].str.replace("RM", "")
df["monthly_rent"] = df["monthly_rent"].str.replace("per month", "")
df["monthly_rent"] = df["monthly_rent"].str.replace(" ", "")
df["monthly_rent"] = pd.to_numeric(df["monthly_rent"], errors="coerce")

# size: "1842 sq.ft." -> "1842" -> 1842
df["size"] = df["size"].str.replace("sq.ft.", "")
df["size"] = pd.to_numeric(df["size"], errors="coerce")

# rooms: "3" -> 3, and "More than 10" -> empty (NaN)
df["rooms"] = pd.to_numeric(df["rooms"], errors="coerce")
```

```
df[["monthly_rent", "size", "rooms"]].head()
```

Out[6]:

	monthly_rent	size	rooms
0	4200.0	1842	5.0
1	2300.0	1170	3.0
2	1000.0	650	3.0
3	1700.0	743	2.0
4	1299.0	494	1.0

3.4 Clean the location

`"Kuala Lumpur - Taman Desa"` becomes `"Taman Desa"`. The region is already in its own column.

In [7]:
```
# "Kuala Lumpur - Taman Desa" -> split at " - " -> take the 2nd part -> "Taman D
df["location"] = df["location"].str.split(" - ").str[1]

# The 10 areas with the most listings
df["location"].value_counts().head(10)
```

Out[7]:
```
location
Cheras            1958
Kajang             900
Shah Alam          900
Setapak            889
Cyberjaya          809
Puchong            758
Sentul             691
Seri Kembangan     682
Kepong             632
Ampang             591
Name: count, dtype: int64
```

3.5 Drop columns we don't use

- `prop_name` – the building name (`ads_id` was already removed in 3.2)
- `completion_year` – almost half is missing
- `facilities` , `additional_facilities` – long text lists

In [8]:
```
# Remove the columns we will not use
df = df.drop(columns=["prop_name", "completion_year", "facilities", "additional_
df.head()
```

Out[8]:

	monthly_rent	location	property_type	rooms	parking	bathroom	size	furnished
0	4200.0	Taman Desa	Condominium	5.0	2.0	6.0	1842	Fully Furnished
1	2300.0	Cheras	Condominium	3.0	1.0	2.0	1170	Partially Furnished
2	1000.0	Taman Desa	Apartment	3.0	NaN	2.0	650	Fully Furnished
3	1700.0	Sentul	Apartment	2.0	1.0	2.0	743	Partially Furnished
4	1299.0	Mont Kiara	Service Residence	1.0	1.0	1.0	494	Not Furnished

3.6 Handle the remaining missing values

- `parking` : fill with the median (1)
- other columns: only a few rows are empty, so we remove them

In [9]:
```
# parking: fill the empty values with the median (the middle value)
df["parking"] = df["parking"].fillna(df["parking"].median())

# Other columns: only a few rows are empty, so we remove those rows
df = df.dropna()

print("Missing values left:", df.isnull().sum().sum())
print("Rows now:", len(df))
```

```
Missing values left: 0
Rows now: 18440
```

3.7 Group rare property types

Types with very few listings (e.g. "Bungalow House") become "Others", because the model can't learn from so few examples.

In [10]:
```
# The property types we want to keep
main_types = ["Condominium", "Service Residence", "Apartment", "Flat", "Studio",

# isin() = "is in the list".  ~ = "NOT".
# So: any type that is NOT in the list becomes "Others"
df.loc[~df["property_type"].isin(main_types), "property_type"] = "Others"

df["property_type"].value_counts()
```

Out[10]:
```
property_type
Condominium          7710
Service Residence    4910
Apartment            4905
Flat                  550
Studio                176
Others                115
Duplex                 74
Name: count, dtype: int64
```

3.8 Find and remove outliers

An **outlier** is a value that is very different from all the others. In this dataset, most outliers are **typing mistakes**, like a rent of RM 2,400,000 per month or a size of 1 sq.ft.

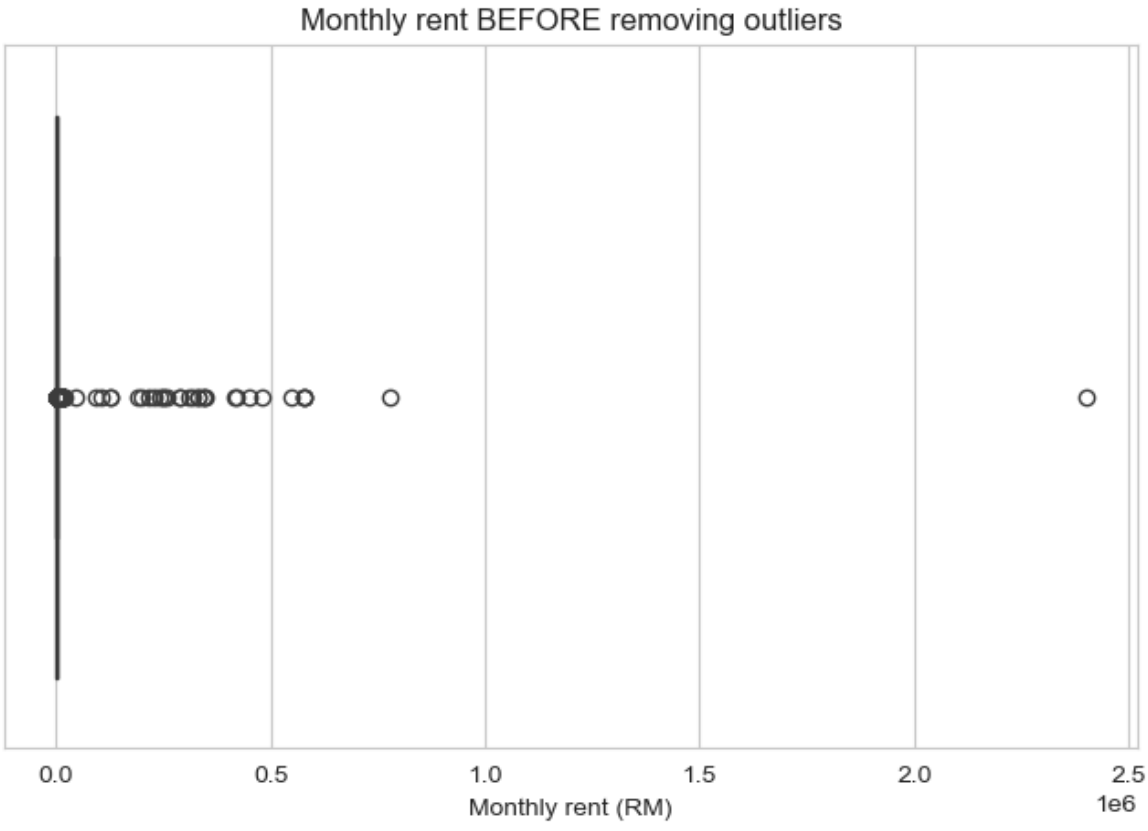
Outliers are a problem because they confuse the model and make the average wrong.

1. **See** the outliers in a boxplot
2. **Look** at the outlier rows
3. **Remove** them
4. **Check** the boxplot again

Step 1: See the outliers in a boxplot

In a boxplot, the **dots outside the whiskers** are outliers.

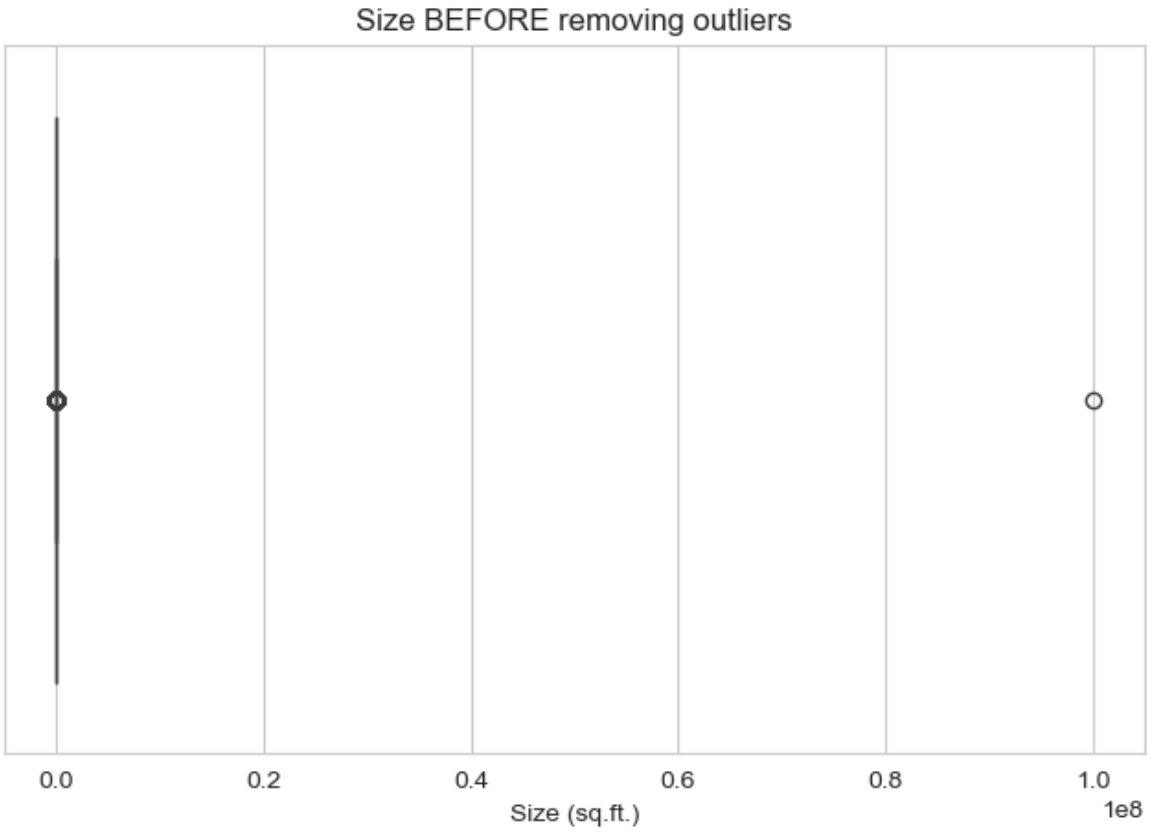
In [11]:
```
sns.boxplot(x=df["monthly_rent"])
plt.title("Monthly rent BEFORE removing outliers")
plt.xlabel("Monthly rent (RM)")
plt.show()
```



How to read the numbers on the bottom: `1e6` at the end of the axis means **× 1,000,000**. So `2.5` on the axis means RM 2,500,000.

What we see: almost all the data is squashed into a thin line on the left, and a few dots are very far to the right. Those dots are rents in the hundreds of thousands, which are impossible for a monthly rent.

In [12]:
```
sns.boxplot(x=df["size"])
plt.title("Size BEFORE removing outliers")
plt.xlabel("Size (sq.ft.)")
plt.show()
```



What we see: the same problem. Here `1e8` means **× 100,000,000**, so the dot at `1.0` is a unit of almost 100 million sq.ft. That is a typing mistake.

Step 2: Look at the outlier rows

Our rule for a sensible listing: rent between **RM 300 and RM 10,000**, and size between **200 and 5,000 sq.ft.** Anything outside this rule is an outlier. The `|` sign means **"or"**.

In [13]:
```
# An outlier breaks at least one of these 4 rules.  | means "or"
outliers = df[(df["monthly_rent"] < 300) | (df["monthly_rent"] > 10000) |
              (df["size"] < 200) | (df["size"] > 5000)]

print("Number of outliers:", len(outliers))
print("Number of listings:", len(df))

# The columns we want to see in the next tables
cols = ["location", "property_type", "monthly_rent", "size"]
```

```
Number of outliers: 182
Number of listings: 18440
```

Now let's look at some of them. `sort_values` puts the rows in order, and `head(5)` shows the first 5.

The 5 highest rents:

In [14]:
```
# Sort from the highest rent, show the first 5
outliers.sort_values("monthly_rent", ascending=False).head(5)[cols]
```

Out[14]:

	location	property_type	monthly_rent	size
4672	Sri Hartamas	Condominium	2400000.0	2573
6367	Bukit Jalil	Condominium	780000.0	971
8640	Bukit Jalil	Condominium	580000.0	1100
2374	Sentul	Condominium	580000.0	1345
5705	Kepong	Service Residence	580000.0	958

The 5 lowest rents:

In [15]:
```
# Sort from the lowest rent, show the first 5
outliers.sort_values("monthly_rent").head(5)[cols]
```

Out[15]:

	location	property_type	monthly_rent	size
18712	Seri Kembangan	Service Residence	70.0	100
8062	Setapak	Condominium	80.0	5
17145	Kota Damansara	Apartment	80.0	400
1716	Setapak	Condominium	90.0	100
10256	Bandar Sri Damansara	Apartment	100.0	400

The 5 biggest sizes:

In [16]:
```
# Sort from the biggest size, show the first 5
outliers.sort_values("size", ascending=False).head(5)[cols]
```

Out[16]:

	location	property_type	monthly_rent	size
1714	Cheras	Condominium	110.0	99999999
2622	Old Klang Road	Apartment	1400.0	48010
4518	Bangsar	Condominium	3300.0	15000
2025	Kepong	Service Residence	1800.0	9001
8703	Cheras	Apartment	1400.0	8300

The 5 smallest sizes:

In [17]:
```
# Sort from the smallest size, show the first 5
outliers.sort_values("size").head(5)[cols]
```

Out[17]:

	location	property_type	monthly_rent	size
19964	Cyberjaya	Service Residence	1200.0	1
17226	Kajang	Apartment	850.0	1
16775	Selayang	Condominium	950.0	1
16320	Petaling Jaya	Condominium	2500.0	1
13635	Shah Alam	Apartment	1300.0	1

- There are **182 outliers**, only about **1%** of the listings, so removing them loses very little data.
- **Rents that are too high:** RM 2,400,000 in Sri Hartamas and RM 780,000 in Bukit Jalil. These are probably the **sale price** typed by mistake.
- **Rents that are too low:** RM 70 or RM 80 per month, which is impossible for a condo.
- **Sizes that are too big:** 99,999,999 sq.ft. in Cheras, which is clearly a typing mistake.
- **Sizes that are too small:** 1 sq.ft., which is also impossible.

Note: a few rents between RM 10,000 and RM 50,000 may be real luxury units. We still remove them, because there are very few (about 30), and our app is for normal tenants.

Step 3: Remove the outliers

In [18]:
```
print("Before:", len(df))

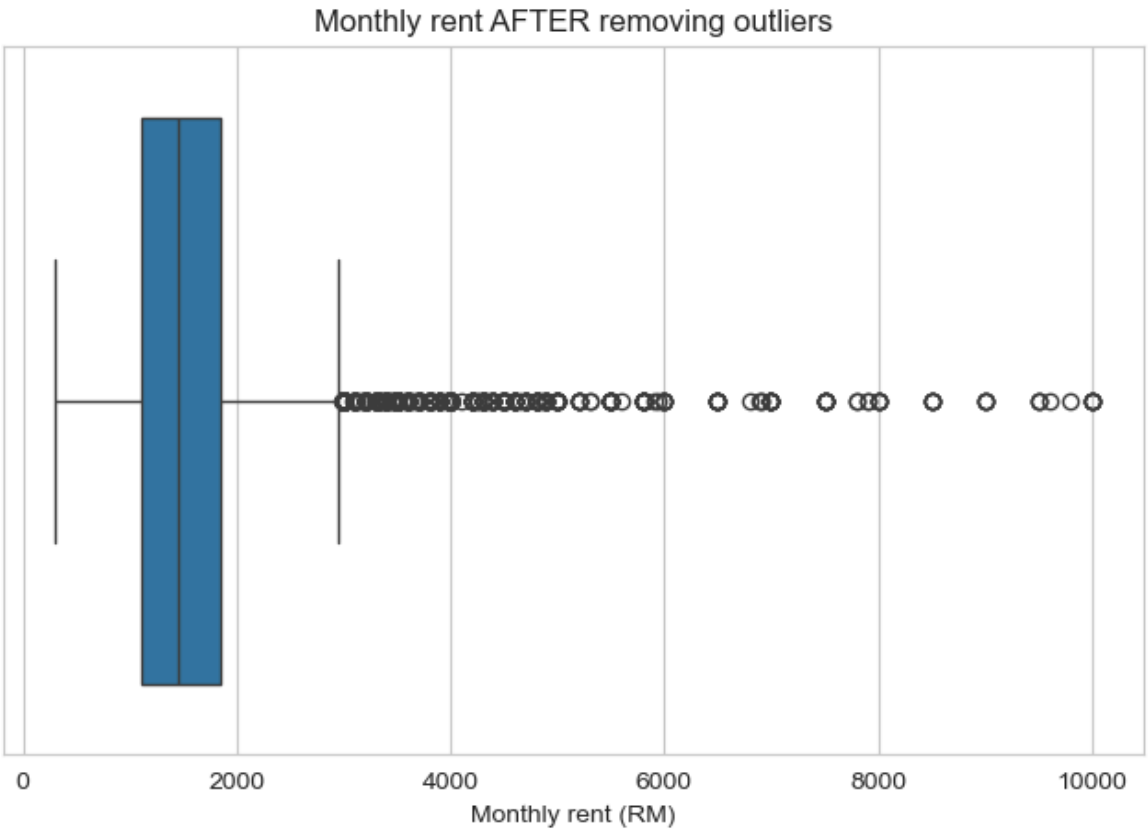
# between(a, b) keeps only the values from a to b
df = df[df["monthly_rent"].between(300, 10000)]
df = df[df["size"].between(200, 5000)]

print("After:", len(df))
```

```
Before: 18440
After: 18258
```

Step 4: Check the boxplot again

In [19]:
```
sns.boxplot(x=df["monthly_rent"])
plt.title("Monthly rent AFTER removing outliers")
plt.xlabel("Monthly rent (RM)")
plt.show()
```



What we see: now we can see the real shape of the data. Most rents are between RM 1,100 and RM 1,850 (the box). The dots on the right are expensive units, but they are realistic, so we keep them.

The average rent also becomes correct: it was **RM 2,297** with the outliers, and now it is **RM 1,610**.

In [20]:
```
# Check the new smallest (min) and biggest (max) values
df[["monthly_rent", "size"]].describe()
```

Out[20]:

	monthly_rent	size
count	18258.000000	18258.000000
mean	1609.739073	918.645963
std	824.357980	304.255176
min	300.000000	200.000000
25%	1100.000000	750.000000
50%	1450.000000	893.000000
75%	1850.000000	1044.000000
max	10000.000000	5000.000000

3.9 Save the cleaned dataset

In [21]:
```
# Save the clean table as a new CSV file (index=False = don't save the row numbe
df.to_csv("cleaned_rent_data.csv", index=False)
```

```
print("Saved cleaned_rent_data.csv")
```

```
Saved cleaned_rent_data.csv
```

Step 4: Exploratory Data Analysis (EDA)

EDA means looking at the clean data with tables and charts **before** building a model.

- **Part A:** What does the data look like?
- **Part B:** What kind of homes are listed, and where?
- **Part C:** What makes rent higher or lower?
- **Part D:** How are the numbers related to each other?

Each chart has a **question** and an **insight** (what we learned).

Part A: What does the data look like?

4.1 Descriptive statistics

`describe()` shows the count, average (mean), smallest (min), biggest (max) and middle values (25%, 50%, 75%) of each number column.

In [22]:
```
df.describe()
```

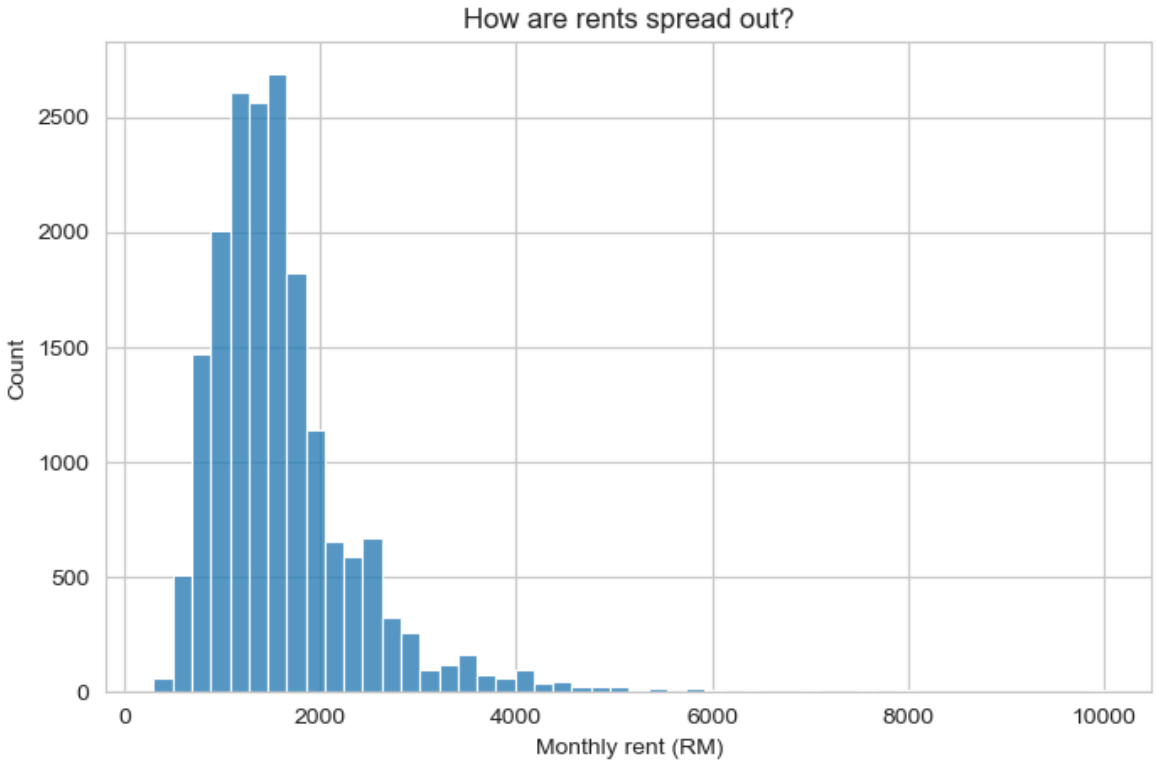
Out[22]:

	monthly_rent	rooms	parking	bathroom	size
count	18258.000000	18258.000000	18258.000000	18258.00000	18258.000000
mean	1609.739073	2.687479	1.295651	1.89369	918.645963
std	824.357980	0.799530	0.507385	0.54830	304.255176
min	300.000000	1.000000	1.000000	1.00000	200.000000
25%	1100.000000	2.000000	1.000000	2.00000	750.000000
50%	1450.000000	3.000000	1.000000	2.00000	893.000000
75%	1850.000000	3.000000	2.000000	2.00000	1044.000000
max	10000.000000	10.000000	10.000000	8.00000	5000.000000

Insight: the median rent is RM 1,450 and the average is RM 1,610. A typical unit is about 900 sq.ft. with 3 rooms, 2 bathrooms and 1 parking. The average is higher than the median because a few very expensive units pull the average up.

4.2 How are rents spread out?

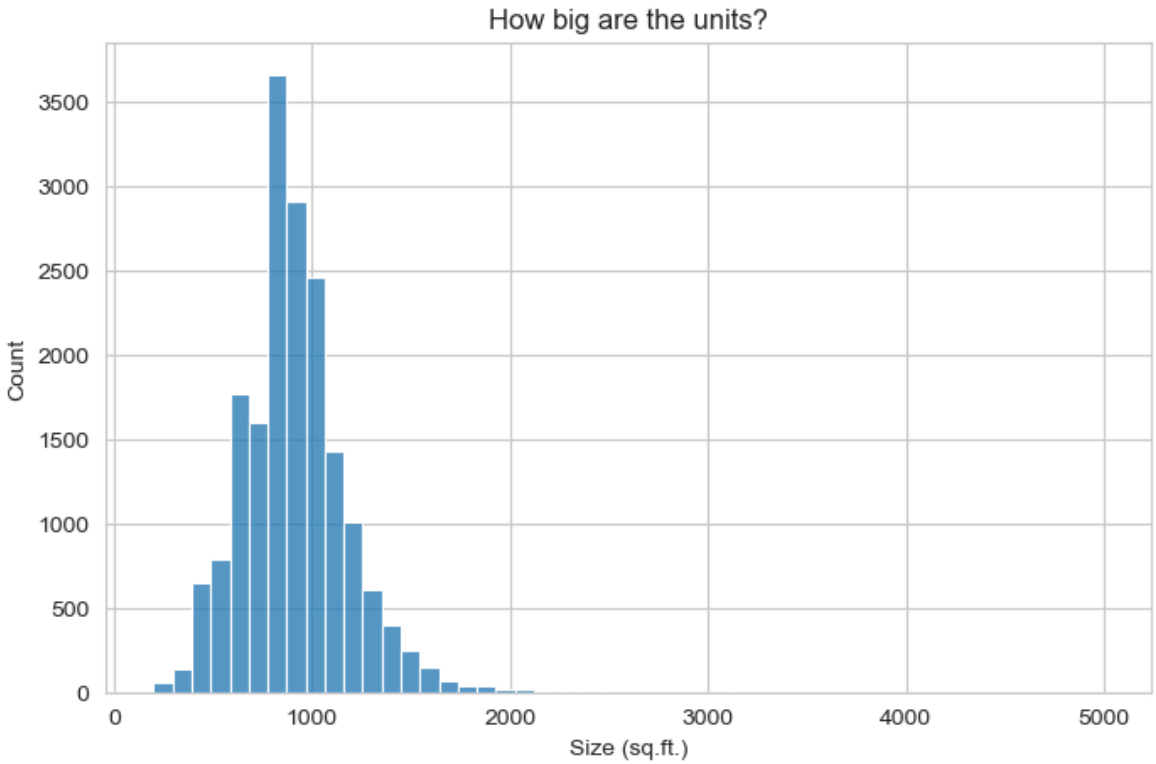
In [23]:
```
sns.histplot(df["monthly_rent"], bins=50)
plt.title("How are rents spread out?")
plt.xlabel("Monthly rent (RM)")
plt.show()
```



Insight: about 65% of units rent for between RM 1,000 and RM 2,000. A few luxury units are much more expensive, which makes a long tail on the right.

4.3 How big are the units?

In [24]:
```
sns.histplot(df["size"], bins=50)
plt.title("How big are the units?")
plt.xlabel("Size (sq.ft.)")
plt.show()
```

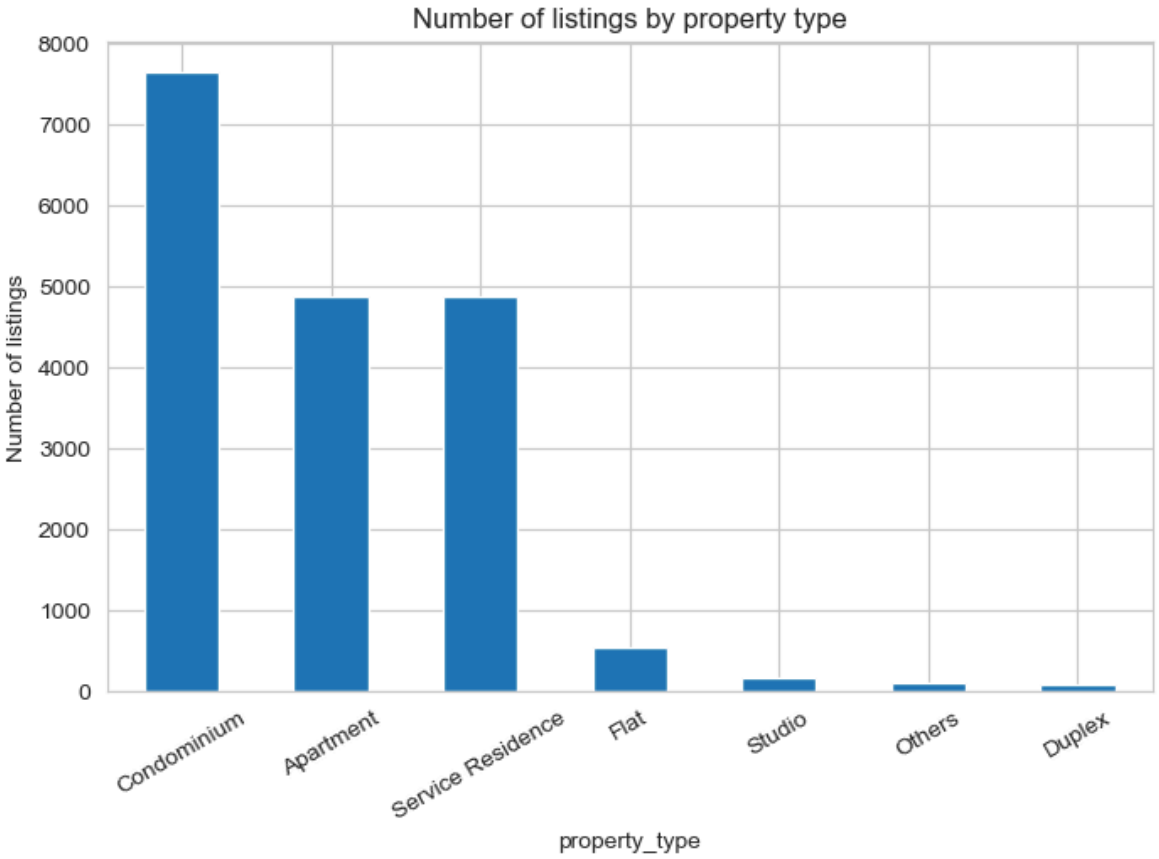


Insight: most units are between 750 and 1,050 sq.ft. (the middle half of the data). Very big units (over 2,000 sq.ft.) are rare.

Part B: What kind of homes are listed, and where?

4.4 Which property types are most common?

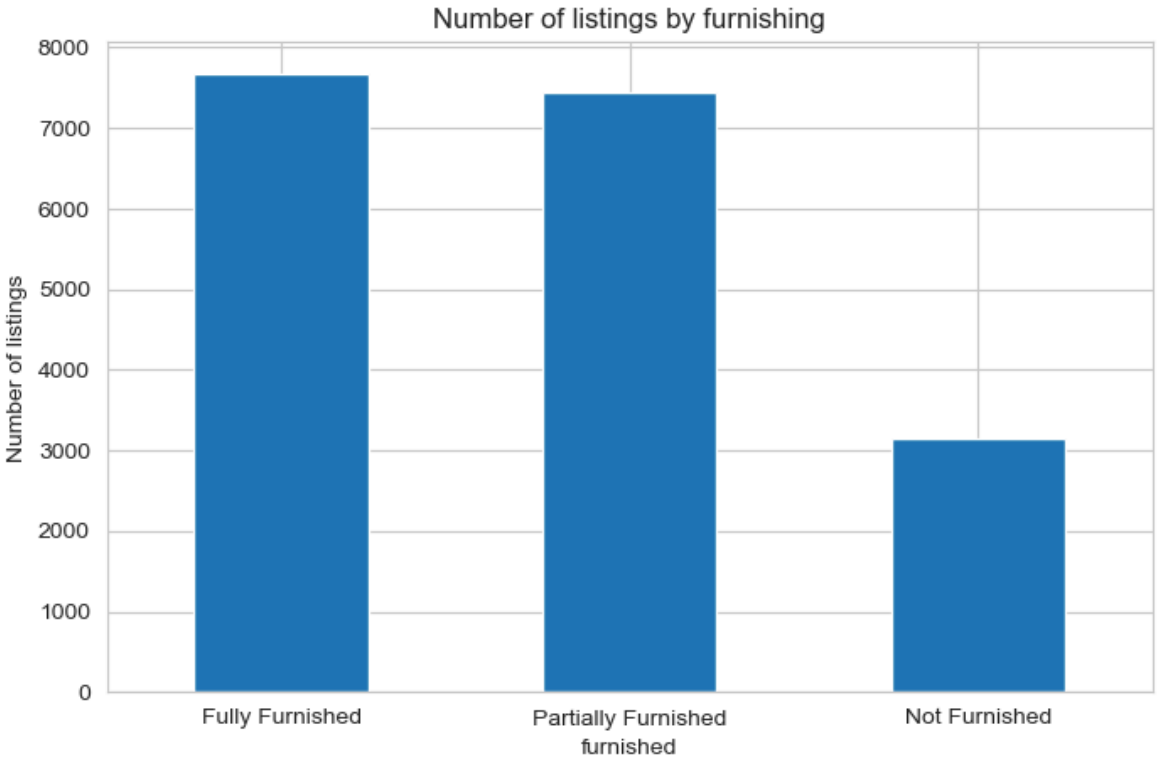
In [25]:
```
df["property_type"].value_counts().plot(kind="bar")
plt.title("Number of listings by property type")
plt.ylabel("Number of listings")
plt.xticks(rotation=30)
plt.show()
```



Insight: Condominiums (7,632), Apartments (4,862) and Service Residences (4,859) are 95% of all listings. Flats, Studios and Duplexes are rare.

4.5 How are the units furnished?

In [26]:
```
df["furnished"].value_counts().plot(kind="bar")
plt.title("Number of listings by furnishing")
plt.ylabel("Number of listings")
plt.xticks(rotation=0)
plt.show()
```

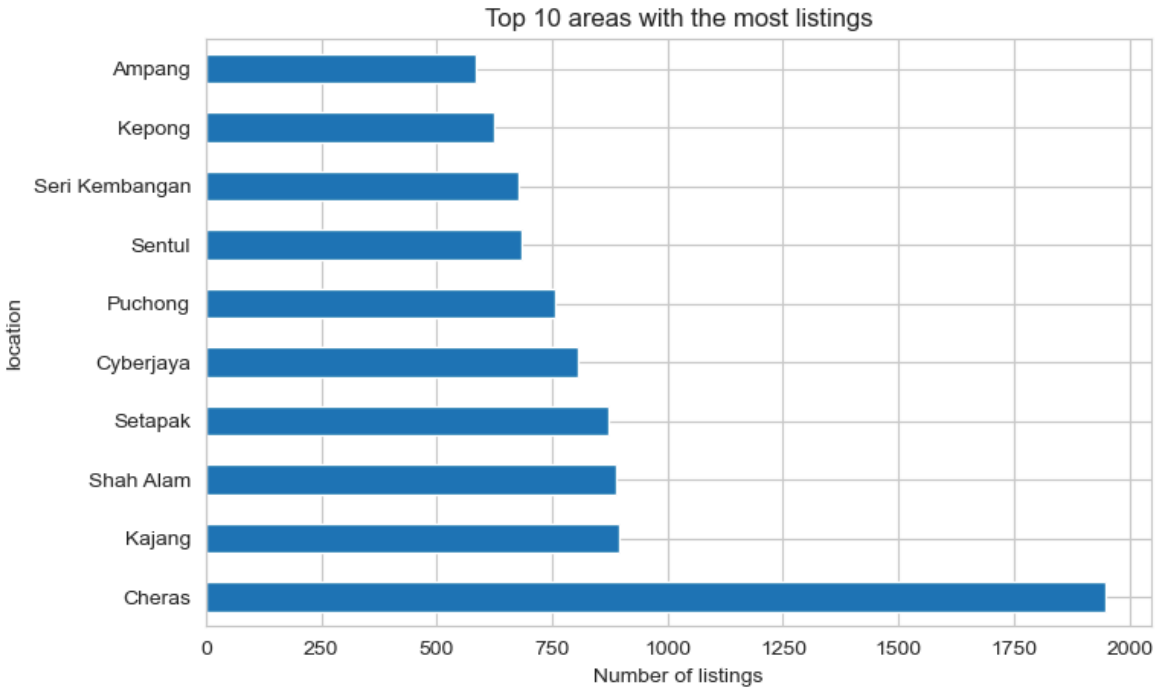


Insight: most units are fully furnished (7,682) or partially furnished (7,436). Only 17% are not furnished (3,140).

4.6 Where are most of the listings?

In [27]:
```
# The 10 areas with the most listings
df["location"].value_counts().head(10).plot(kind="barh")
plt.title("Top 10 areas with the most listings")
plt.xlabel("Number of listings")
plt.show()

# Number of listings in KL and in Selangor
print(df["region"].value_counts())
```



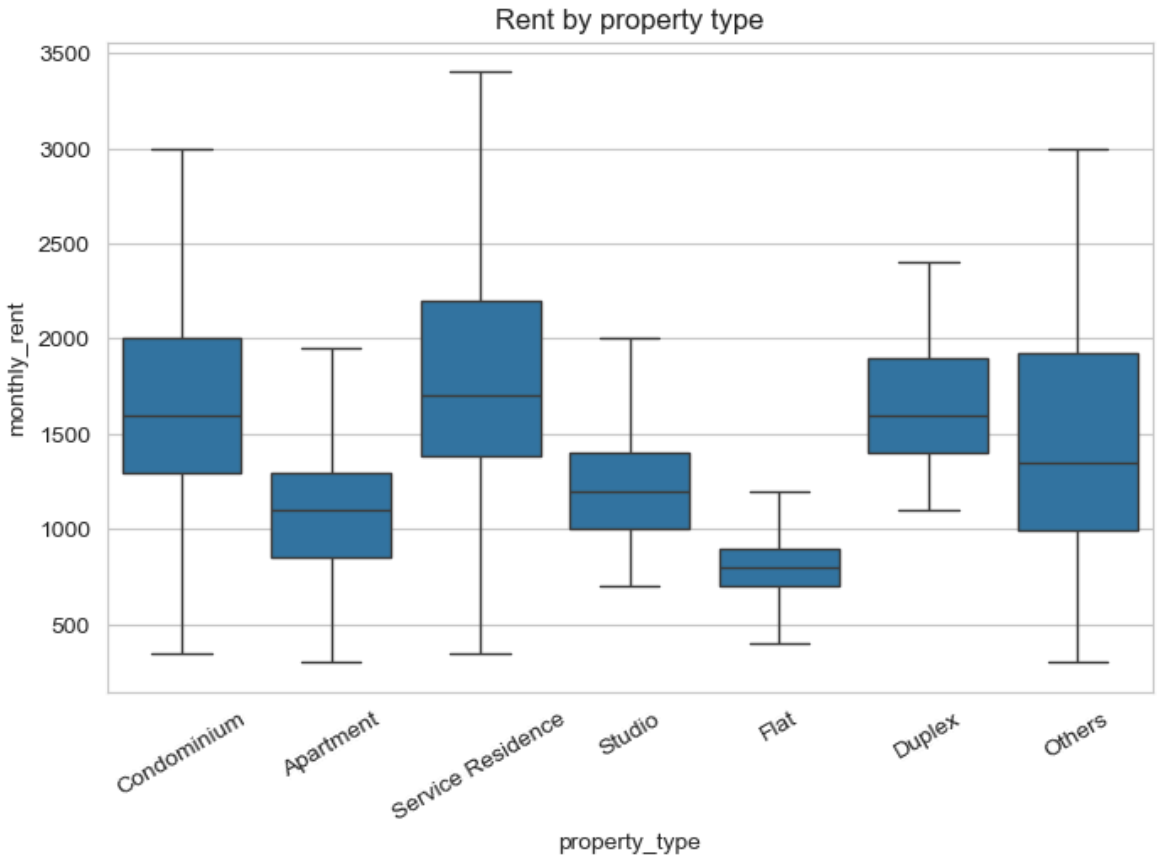
```
region
Selangor        9148
Kuala Lumpur    9110
Name: count, dtype: int64
```

Insight: Cheras has by far the most listings (1,949), more than double Kajang (894). KL and Selangor have almost the same number of listings (9,110 and 9,148), so both regions are well represented.

Part C: What makes rent higher or lower?

4.7 Rent by property type

In [28]:
```
# showfliers=False = hide the outlier dots, so the boxes are easier to see
sns.boxplot(data=df, x="property_type", y="monthly_rent", showfliers=False)
plt.title("Rent by property type")
plt.xticks(rotation=30)
plt.show()
```

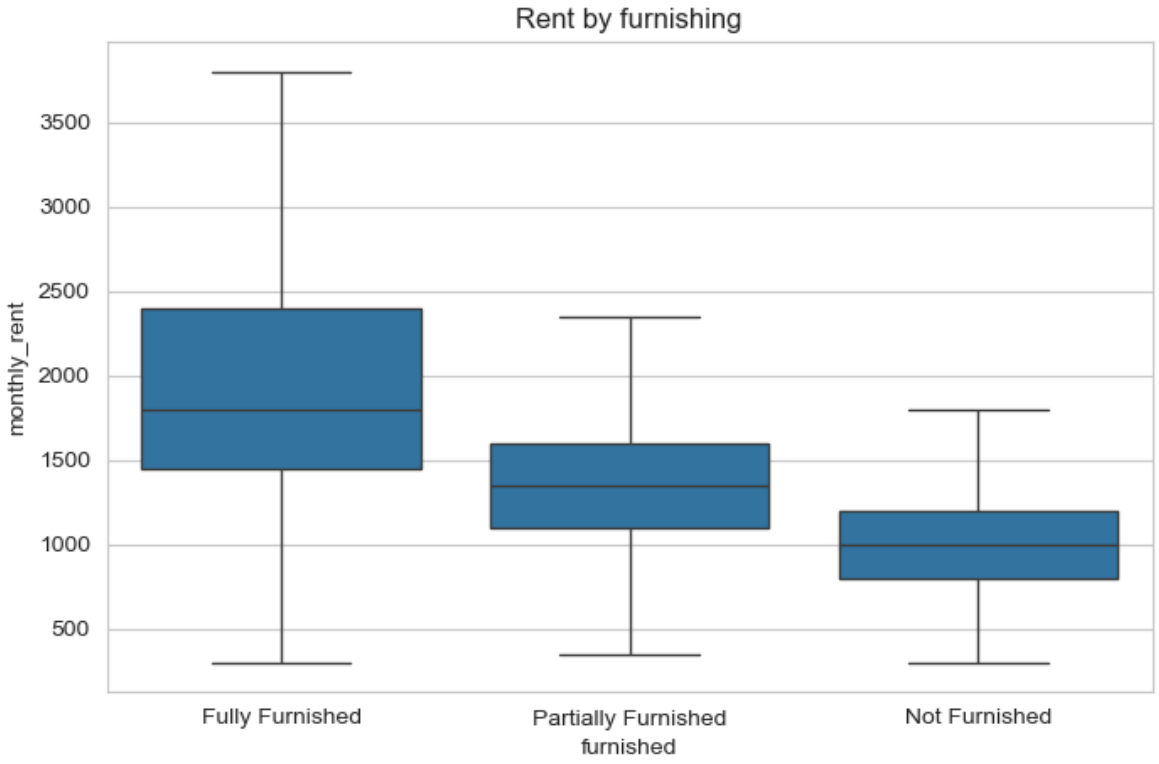


How to read a boxplot: the line in the middle of the box is the median. The box holds the middle half of the rents.

Insight: Flats are the cheapest (median RM 800). Condominiums (RM 1,600) and Service Residences (RM 1,700) are the most expensive.

4.8 Rent by furnishing

In [29]:
```
sns.boxplot(data=df, x="furnished", y="monthly_rent", showfliers=False)
plt.title("Rent by furnishing")
plt.show()
```

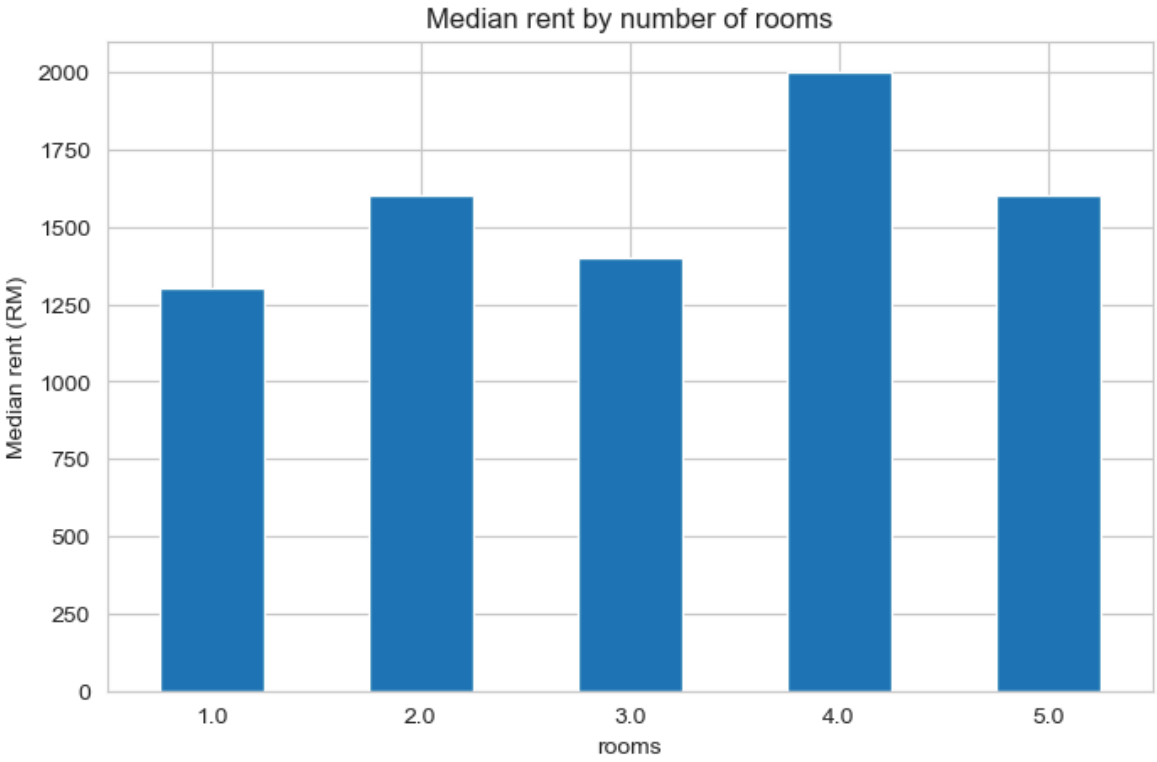


Insight: fully furnished units (median RM 1,800) cost almost double not furnished units (RM 1,000).

4.9 Does more rooms mean higher rent?

We only show 1 to 5 rooms, because 6 or more rooms have very few listings.

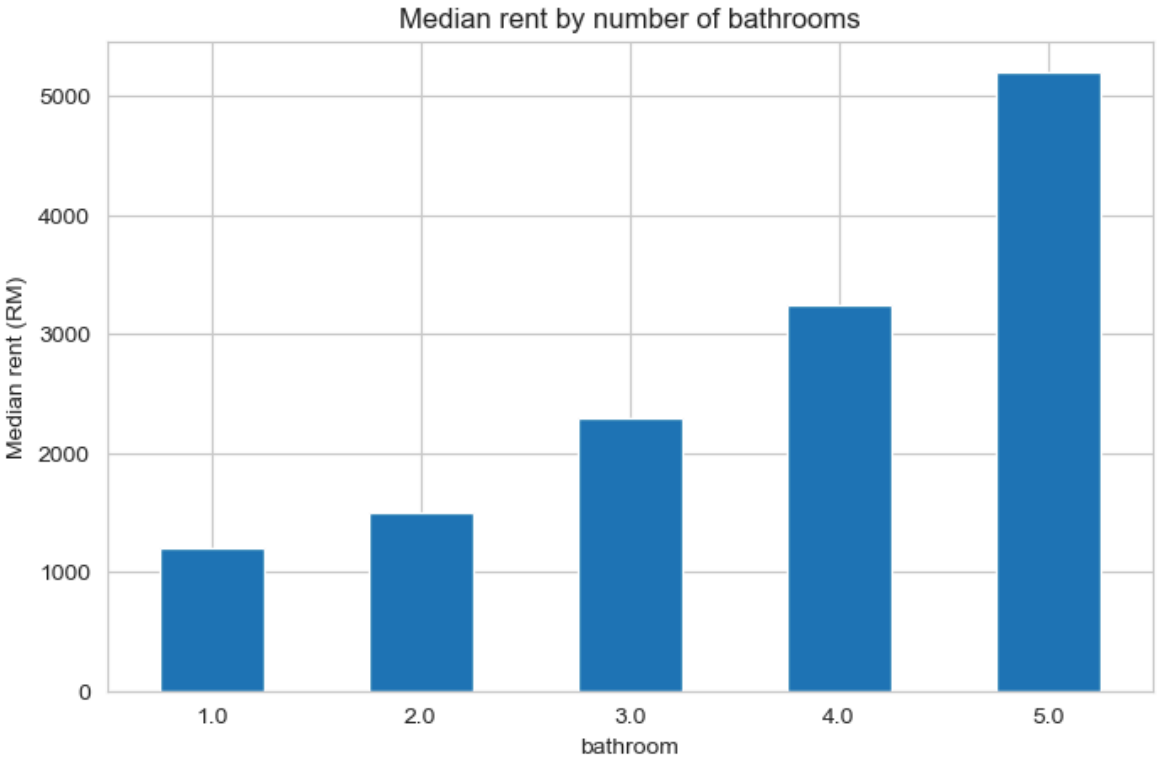
In [30]:
```
# Keep 1 to 5 rooms -> group by rooms -> take the median rent of each group
df[df["rooms"] <= 5].groupby("rooms")["monthly_rent"].median().plot(kind="bar")
plt.title("Median rent by number of rooms")
plt.ylabel("Median rent (RM)")
plt.xticks(rotation=0)
plt.show()
```



Insight: not always. The bars go up and down: 2-room units (median RM 1,600) cost more than 3-room units (RM 1,400), and 5-room units (RM 1,600) cost less than 4-room units (RM 2,000). Many 2-room units are fully furnished Service Residences, while many 3-room units are partially furnished Apartments. So the **type and furnishing** matter more than the number of rooms.

4.10 Does more bathrooms mean higher rent?

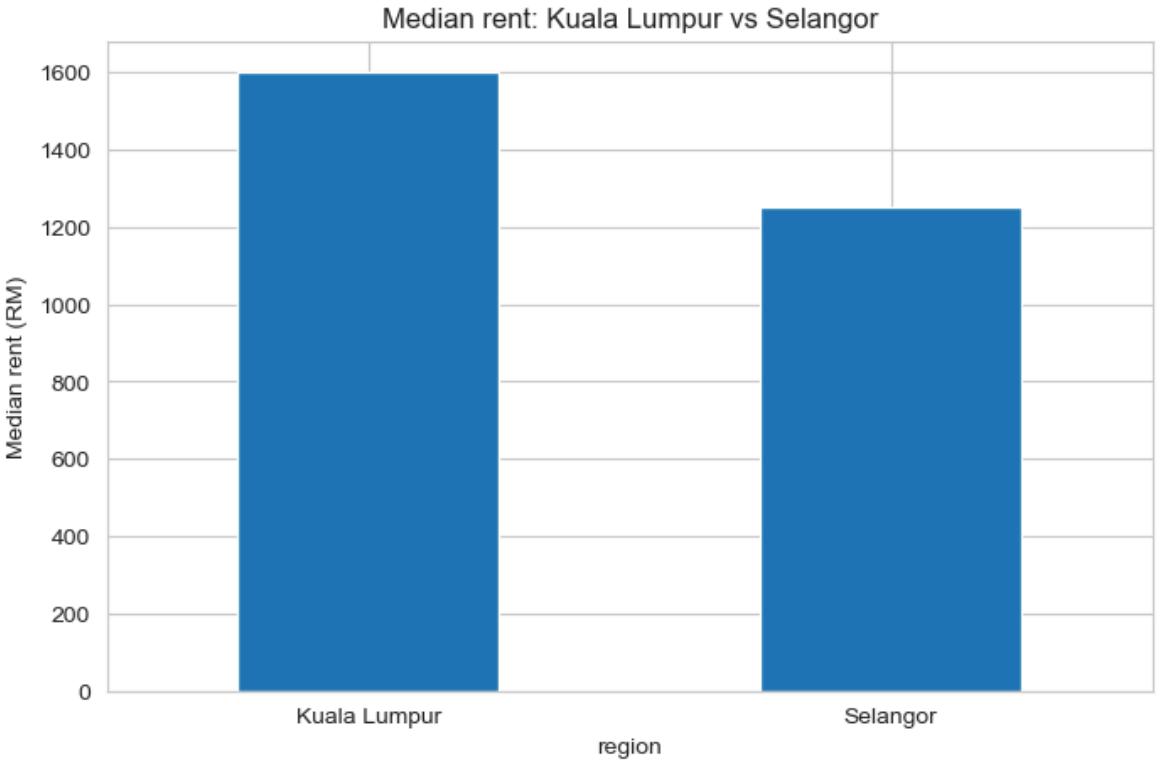
In [31]:
```
# Keep 1 to 5 bathrooms -> group by bathroom -> take the median rent of each gro
df[df["bathroom"] <= 5].groupby("bathroom")["monthly_rent"].median().plot(kind="
plt.title("Median rent by number of bathrooms")
plt.ylabel("Median rent (RM)")
plt.xticks(rotation=0)
plt.show()
```



Insight: yes, very clearly. Median rent goes up with every extra bathroom: RM 1,200 (1), RM 1,498 (2), RM 2,300 (3), RM 3,250 (4) and RM 5,200 (5). More bathrooms usually means a bigger, higher-end unit.

4.11 Kuala Lumpur vs Selangor

In [32]:
```
# Group by region -> take the median rent of each region
df.groupby("region")["monthly_rent"].median().plot(kind="bar")
plt.title("Median rent: Kuala Lumpur vs Selangor")
plt.ylabel("Median rent (RM)")
plt.xticks(rotation=0)
plt.show()
```



Insight: Kuala Lumpur (median RM 1,600) is more expensive than Selangor (RM 1,250).

4.12 Most expensive and cheapest areas

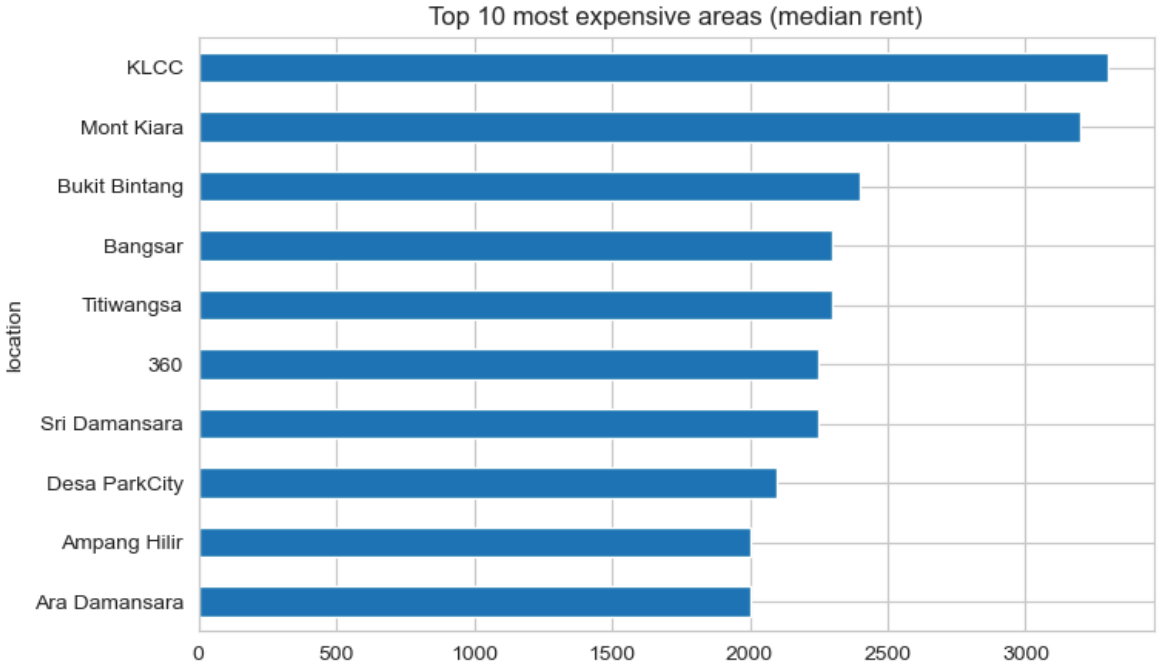
only use areas with at least 30 listings, so the result is reliable.

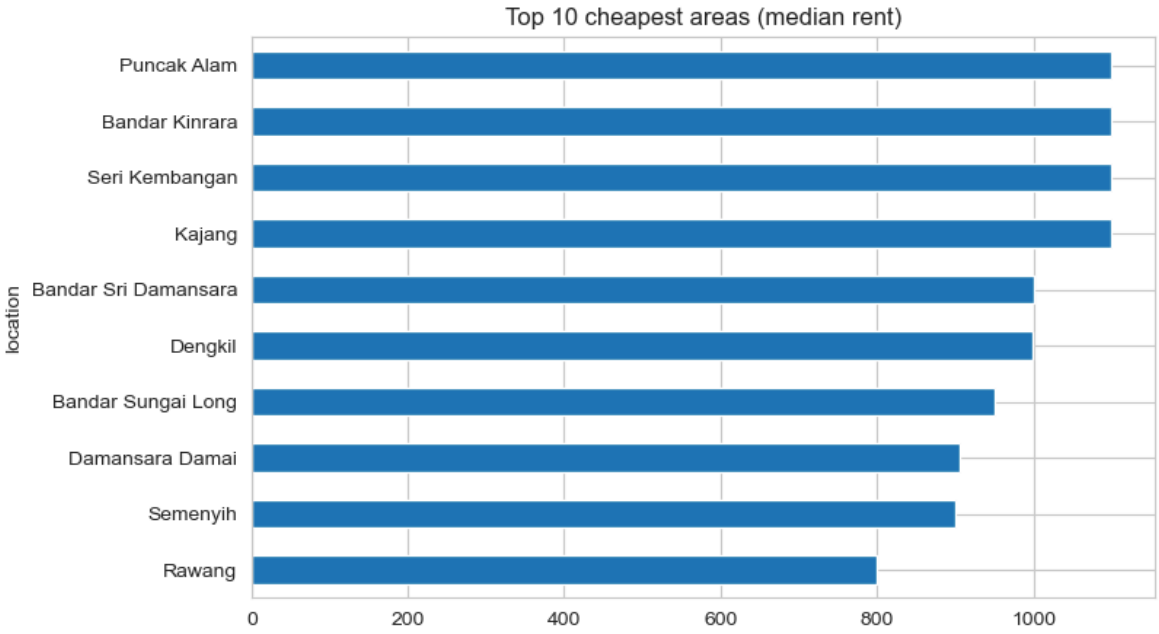
In [33]:
```
# Keep only areas with at least 30 listings
area_counts = df["location"].value_counts()
big_areas = area_counts[area_counts >= 30].index
df_big = df[df["location"].isin(big_areas)]

# Median rent of each area, from cheapest to most expensive
area_rent = df_big.groupby("location")["monthly_rent"].median().sort_values()

area_rent.tail(10).plot(kind="barh")
plt.title("Top 10 most expensive areas (median rent)")
plt.show()

area_rent.head(10).plot(kind="barh")
plt.title("Top 10 cheapest areas (median rent)")
plt.show()
```



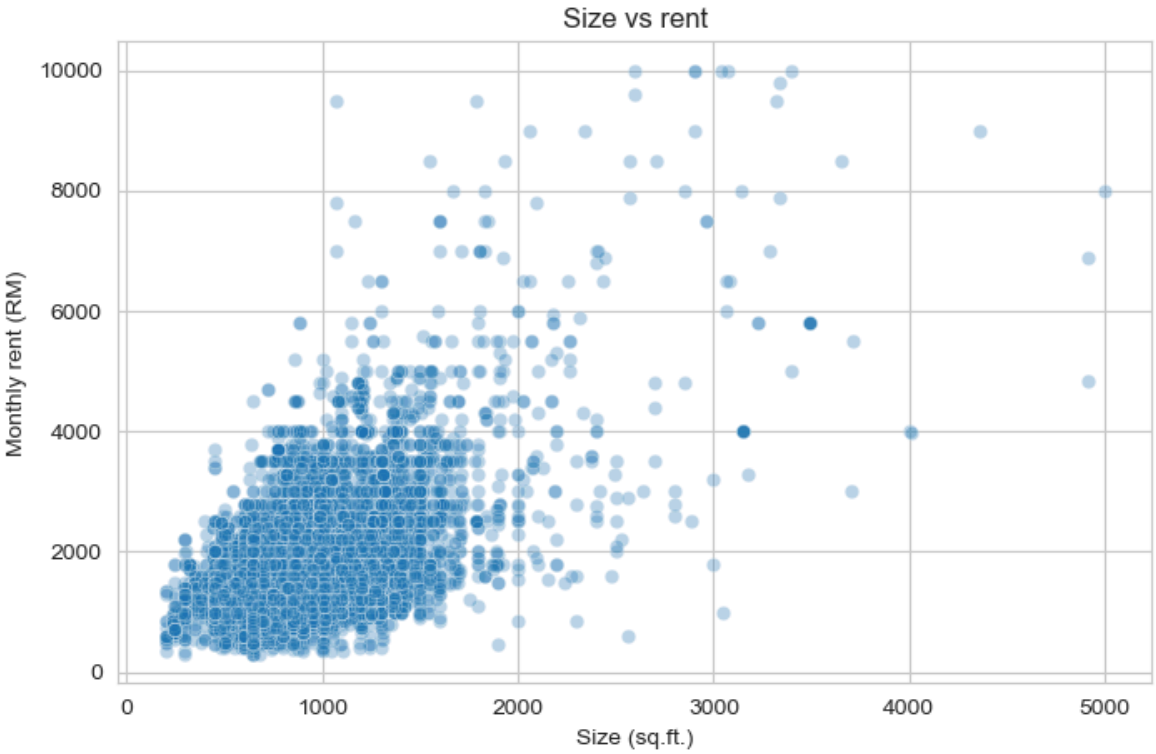


Insight: KLCC (RM 3,300) and Mont Kiara (RM 3,200) are the most expensive. Rawang (RM 800) and Semenyih (RM 900) are the cheapest. The same kind of unit can cost about **4 times more** depending on the area, so location is a very strong factor.

Part D: How are the numbers related?

4.13 Size vs rent

In [34]:
```
# Each dot is one listing. alpha=0.3 makes the dots see-through
sns.scatterplot(data=df, x="size", y="monthly_rent", alpha=0.3)
plt.title("Size vs rent")
plt.xlabel("Size (sq.ft.)")
plt.ylabel("Monthly rent (RM)")
plt.show()
```

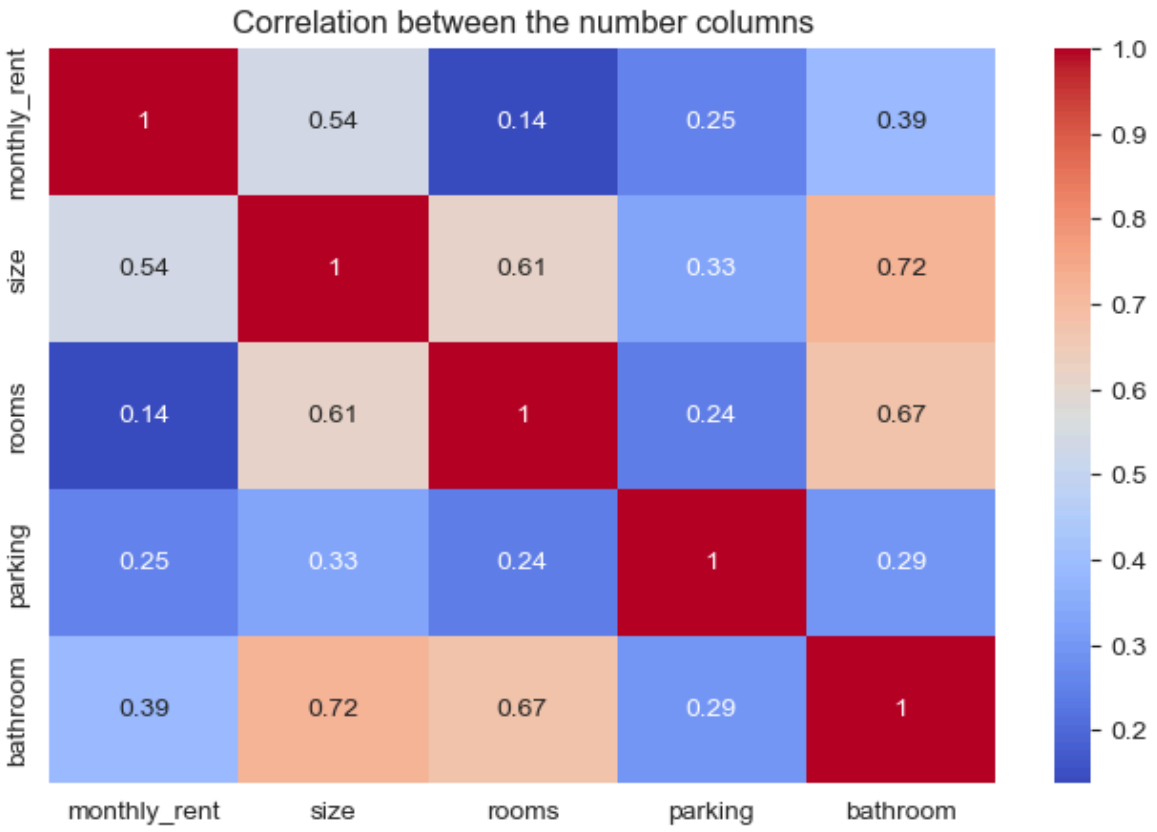


Insight: bigger units usually cost more (the dots go up to the right), but the dots are very spread out, because location and furnishing also change the price a lot.

4.14 Correlation heatmap

Correlation shows how strongly two numbers move together: **1** = always together, **0** = no link.

In [35]:
```
# corr() = correlation between the number columns. annot=True writes the numbers
sns.heatmap(df[["monthly_rent", "size", "rooms", "parking", "bathroom"]].corr(),
plt.title("Correlation between the number columns")
plt.show()
```



Insight: size has the strongest link with rent (0.54), then bathroom (0.39) and parking (0.25). Rooms has a weak link (0.14), which matches what we saw in 4.9.

EDA summary

#	Finding	Chart
1	Most rents are between RM 1,000 and RM 2,000	4.2
2	Condominiums, Apartments and Service Residences are 95% of listings	4.4
3	**Location** is the strongest factor: KLCC costs about 4× more than Rawang	4.12
4	**Furnishing**: fully furnished costs almost double not furnished	4.8
5	**Size** and **bathrooms** push rent up; rooms alone do not	4.9, 4.10, 4.14
6	Kuala Lumpur is more expensive than Selangor	4.11

Why this matters for the model: rent depends on many things at the same time (area + furnishing + size), and not in a straight line. That is why we expect **Random Forest** to do better than Linear Regression in Step 6.

Step 5: Prepare the data

5.1 Features (X) and target (y)

- **X** = the information we give the model
- **y** = what we want to predict (monthly rent)

In [36]:
```
# X = the information we give the model (all columns except the rent)
X = df.drop(columns=["monthly_rent"])

# y = what we want to predict (the rent)
y = df["monthly_rent"]
```

5.2 Encode the text columns

Models only understand numbers. `get_dummies` turns each text option into a 0/1 column.

In [37]:
```
# Turn text columns into 0/1 columns (e.g. location_Cheras = 1 or 0)
X = pd.get_dummies(X, drop_first=True)

print("Number of columns after encoding:", X.shape[1])
X.head()
```

```
Number of columns after encoding: 137
```

Out[37]:

	rooms	parking	bathroom	size	location_369	location_389	location_517	location_
0	5.0	2.0	6.0	1842	False	False	False	F
1	3.0	1.0	2.0	1170	False	False	False	F
2	3.0	1.0	2.0	650	False	False	False	F
3	2.0	1.0	2.0	743	False	False	False	F
4	1.0	1.0	1.0	494	False	False	False	F

5.3 Train/test split (80% / 20%)

The model learns from 80% of the data and is tested on the other 20%, which it has never seen. `random_state=42` makes the split the same every time.

In [38]:
```
# 80% to train (learn), 20% to test. random_state=42 = the same split every time
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

```
Training rows: 14606
Testing rows: 3652
```

5.4 Scale the data

Scaling puts all numbers on a similar range, which helps Linear Regression. We `fit` the scaler on the training data only, so no information from the test data leaks into training (no data leakage).

In [39]:
```
scaler = StandardScaler()

# fit on the TRAINING data only (to avoid data leakage), then scale it
X_train_scaled = scaler.fit_transform(X_train)

# scale the test data with the same scaler (no fit!)
X_test_scaled = scaler.transform(X_test)
```

Step 6: Train the models

- **Linear Regression** – finds the best straight-line relationship between the features and rent. A simple baseline.
- **Random Forest** – builds many decision trees (each asks yes/no questions like "Is size > 1000?") and averages their answers. It can learn complex patterns, like location changing the price a lot.

6.1 Linear Regression

In [40]:
```
# 1. Create the model
lr = LinearRegression()

# 2. Train it on the training data
lr.fit(X_train_scaled, y_train)

# 3. Predict the rent of the test data
lr_pred = lr.predict(X_test_scaled)

# 4. Compare the predictions with the real rents
lr_mae = mean_absolute_error(y_test, lr_pred)
lr_mse = mean_squared_error(y_test, lr_pred)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_pred)

print("MAE : RM", round(lr_mae))
print("MSE :", round(lr_mse))
print("RMSE: RM", round(lr_rmse))
print("R2  :", round(lr_r2, 3))
```

```
MAE : RM 310
MSE : 236850
RMSE: RM 487
R2  : 0.665
```

6.2 Random Forest

`max_features=0.33` means each question in a tree only looks at a random third of the columns. This makes the trees more different from each other, so more of them learn from the area columns (location), which is very important for rent.

In [41]:
```
# 1. Create the model (100 trees by default)
rf = RandomForestRegressor(max_features=0.33, random_state=42)

# 2. Train it
rf.fit(X_train_scaled, y_train)
```

```
# 3. Predict the rent of the test data
rf_pred = rf.predict(X_test_scaled)

# 4. Compare the predictions with the real rents
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

print("MAE : RM", round(rf_mae))
print("MSE :", round(rf_mse))
print("RMSE: RM", round(rf_rmse))
print("R2  :", round(rf_r2, 3))
```

```
MAE : RM 217
MSE : 157841
RMSE: RM 397
R2  : 0.777
```

Step 7: Evaluation

7.1 Compare the models

In [42]:
```
# Put the results of both models in one table
results = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest"],
    "MAE":   [lr_mae, rf_mae],
    "MSE":   [lr_mse, rf_mse],
    "RMSE":  [lr_rmse, rf_rmse],
    "R2":    [lr_r2, rf_r2],
})
results.round(3)
```

Out[42]:

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	310.349	236850.083	486.672	0.665
1	Random Forest	216.797	157841.063	397.292	0.777

What the metrics mean

- **MAE** – the average mistake, in RM
- **MSE** – the average of squared mistakes (big mistakes count more)
- **RMSE** – the square root of MSE, back in RM
- **R^2** – how much of the rent the model explains (1 = perfect, 0 = useless)

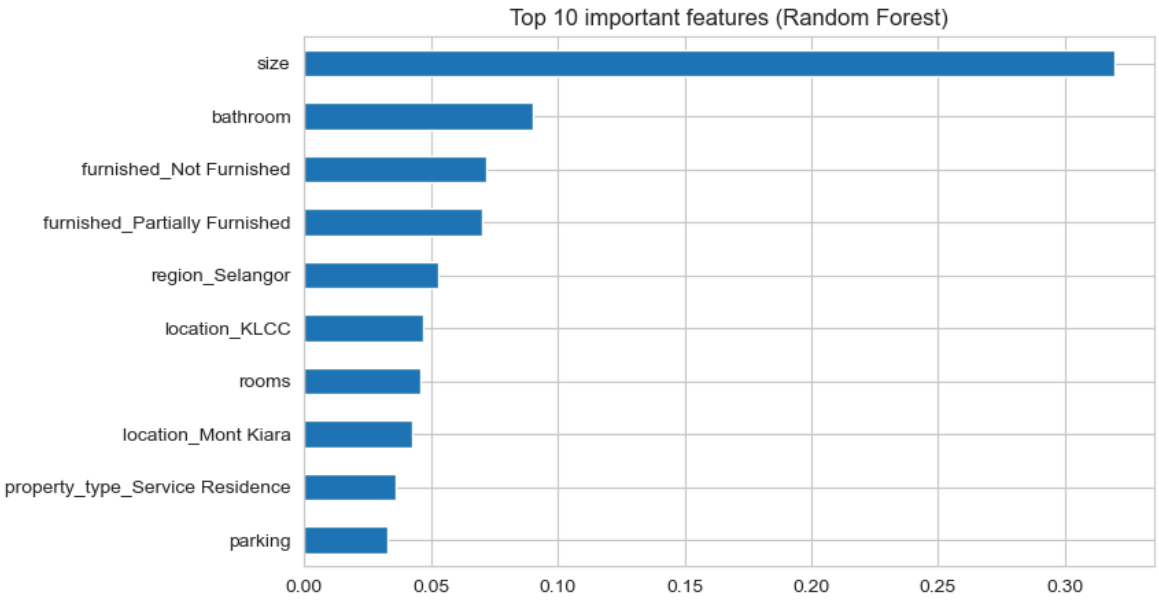
Interpretation

- **Random Forest is the best model.** On average, its prediction is about **RM 217** away from the real rent, and it explains about **78%** of the rent (R^2 = 0.78).
- Linear Regression is worse (MAE RM 310, R^2 = 0.66), because the link between the features and rent is not a straight line.
- RMSE (RM 397) is bigger than MAE, so there are still some big mistakes, mostly on expensive luxury units.

7.2 Most important features

In [43]:
```
# How much each column helped the Random Forest (bigger = more important)
importance = pd.Series(rf.feature_importances_, index=X.columns)

# Show the 10 most important columns
importance.sort_values().tail(10).plot(kind="barh")
plt.title("Top 10 important features (Random Forest)")
plt.show()
```



Insight: size is by far the most important feature, then bathrooms, furnishing, region (KL or Selangor) and expensive areas like KLCC and Mont Kiara. This matches the EDA.

Step 8: Try a prediction

We give the model one new property, encode it the same way, scale it, and predict.

In [44]:
```
# 1. A new house that the model has never seen
new_house = pd.DataFrame([{
    "location": "Cheras",
    "property_type": "Condominium",
    "rooms": 3,
    "parking": 1,
    "bathroom": 2,
    "size": 1000,
    "furnished": "Fully Furnished",
    "region": "Kuala Lumpur",
}])

# 2. Encode it the same way as the training data
new_house = pd.get_dummies(new_house)

# 3. Make sure it has the same columns, in the same order (missing columns = 0)
new_house = new_house.reindex(columns=X.columns, fill_value=0)

# 4. Scale it with the same scaler
new_house = scaler.transform(new_house)
```

```
# 5. Predict
print("Predicted rent: RM", round(rf.predict(new_house)[0]))
```

```
Predicted rent: RM 1584
```

Step 9: Classification with ROC and AUC

ROC and AUC only work when we predict a **group**, not a number. So we ask a new question:

Is the property Expensive (1) or Affordable (0)? Expensive = rent above the median rent of the training data.

In [45]:
```
# The median rent of the training data is our line between the 2 groups
threshold = y_train.median()
print("Expensive = rent above RM", threshold)

# True/False -> astype(int) -> 1/0.   1 = Expensive, 0 = Affordable
y_train_class = (y_train > threshold).astype(int)
y_test_class = (y_test > threshold).astype(int)
```

```
Expensive = rent above RM 1450.0
```

9.1 Train two classification models

- **Logistic Regression** – gives the probability that a property is Expensive
- **Random Forest Classifier** – many decision trees vote Expensive or Affordable

In [46]:
```
# Model 1: Logistic Regression (max_iter=1000 gives it enough steps to learn)
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train_class)

# Model 2: Random Forest Classifier
rf_class = RandomForestClassifier(random_state=42)
rf_class.fit(X_train_scaled, y_train_class)
```

Out[46]:

▾ RandomForestClassifier

▶ Parameters

9.2 Accuracy, precision, recall and F1

- **Accuracy** – how many predictions are correct
- **Precision** – when the model says Expensive, how often it is right
- **Recall** – how many of the truly Expensive units it found
- **F1** – a balance of precision and recall

In [47]:
```
# classification_report shows precision, recall, f1-score and accuracy
print("Logistic Regression")
print(classification_report(y_test_class, log_model.predict(X_test_scaled)))
```

```
print("Random Forest")
print(classification_report(y_test_class, rf_class.predict(X_test_scaled)))
```

```
Logistic Regression
              precision    recall  f1-score   support

           0       0.84      0.85      0.84      1816
           1       0.85      0.84      0.85      1836

    accuracy                           0.85      3652
   macro avg       0.85      0.85      0.85      3652
weighted avg       0.85      0.85      0.85      3652

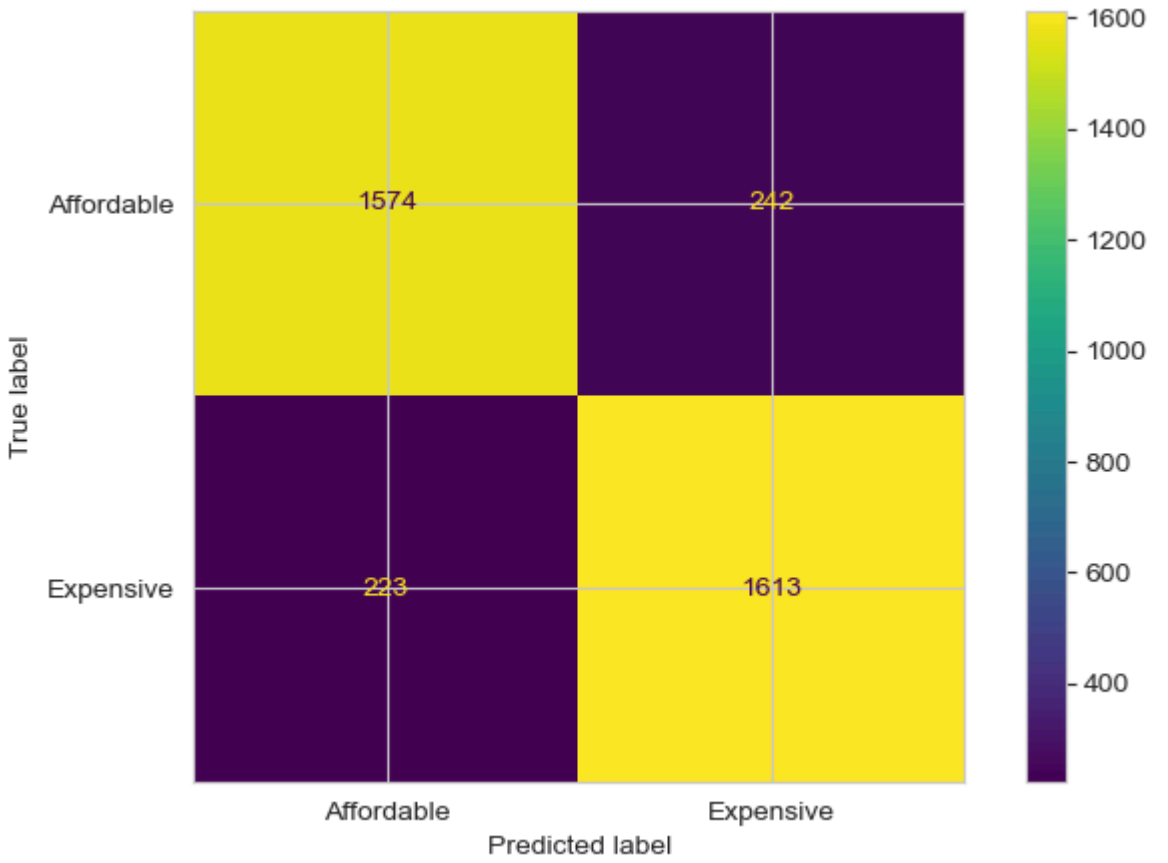
Random Forest
              precision    recall  f1-score   support

           0       0.88      0.87      0.87      1816
           1       0.87      0.88      0.87      1836

    accuracy                           0.87      3652
   macro avg       0.87      0.87      0.87      3652
weighted avg       0.87      0.87      0.87      3652
```

9.3 Confusion matrix (Random Forest)

In [48]:
```
# Compare the real groups with the predicted groups
ConfusionMatrixDisplay.from_predictions(y_test_class, rf_class.predict(X_test_sc
                                        display_labels=["Affordable", "Expensive
plt.show()
```



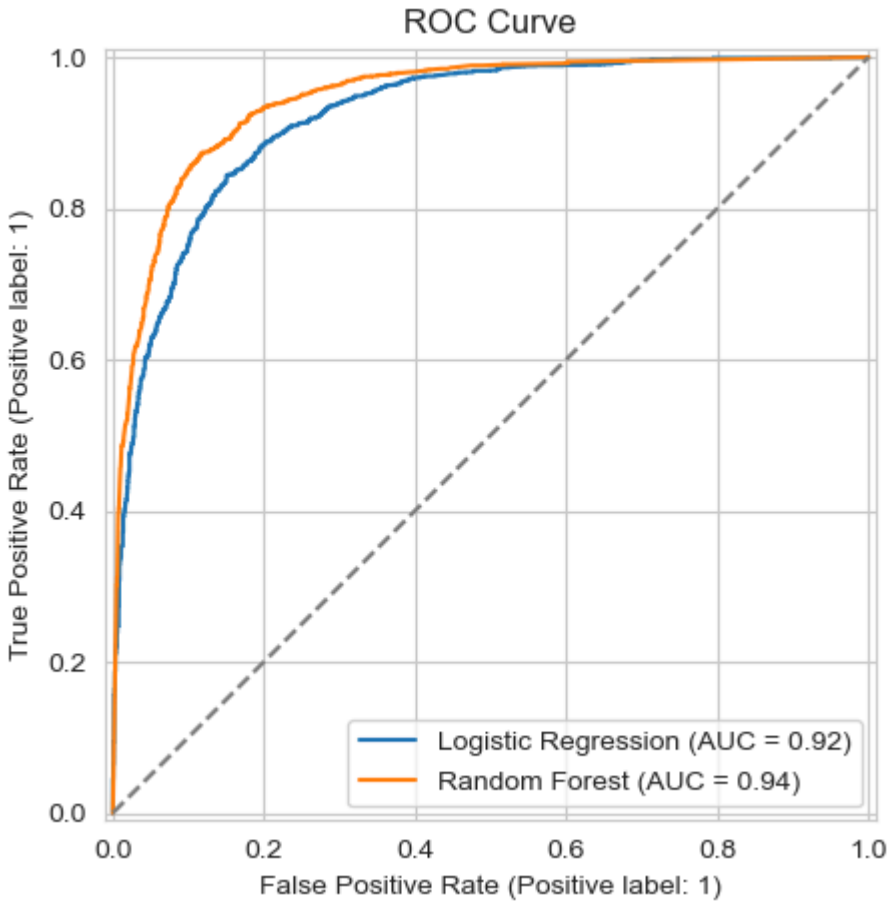
The diagonal boxes (top-left and bottom-right) are correct predictions. The other two boxes are mistakes.

9.4 ROC curve and AUC

- The ROC curve shows how well the model separates Expensive and Affordable.
- The closer the curve is to the top-left corner, the better.
- **AUC**: 1.0 = perfect, 0.5 = random guessing.

In [49]:
```
# ax = the chart. We draw both ROC curves on the SAME chart
ax = plt.gca()
RocCurveDisplay.from_estimator(log_model, X_test_scaled, y_test_class, name="Log
RocCurveDisplay.from_estimator(rf_class, X_test_scaled, y_test_class, name="Rand

# The grey dashed line = random guessing (AUC 0.5)
plt.plot([0, 1], [0, 1], "--", color="grey")
plt.title("ROC Curve")
plt.show()
```



Interpretation

- Both curves are far above the grey line (random guessing), so both models separate Expensive and Affordable units well.
- **Random Forest has the higher AUC (0.94)**, compared with 0.92 for Logistic Regression.
- Random Forest accuracy is 87%: about 9 out of 10 units are put in the correct group.

Step 10: Export the model for the React web app (Houses Rental)

A web browser cannot run Python. So we save a Random Forest as `model.json`, a text file that JavaScript can read.

The full Random Forest is too big for a website, so we train a **small one**:

- `n_estimators=30` → 30 trees instead of 100
- `min_samples_leaf=3` → each final answer is based on at least 3 listings (this keeps the trees smaller)
- `max_features=0.33` → the same setting as our main Random Forest

Trees don't need scaled data, so we use `X_train`.

In [50]:
```
# A smaller Random Forest: 30 trees, so the file is small enough for a website.
# Trees don't need scaled data, so we use X_train (not scaled)
web_rf = RandomForestRegressor(n_estimators=30, min_samples_leaf=3, max_features
web_rf.fit(X_train, y_train)

web_mae = mean_absolute_error(y_test, web_rf.predict(X_test))
print("Small model - average error (MAE): RM", round(web_mae))
```

```
Small model - average error (MAE): RM 240
```

Each tree is saved as 5 lists:

- `feature` – which column the tree checks (e.g. size)
- `threshold` – the number it compares with (e.g. 875.5)
- `left` / `right` – where to go next (-1 means the end)
- `value` – the predicted rent at the end

We also save:

- for each area: its region (so the website fills it in by itself), its median rent and its number of listings
- market numbers (median rent by region, property type and furnishing) for the dashboard page

In [51]:
```
# 1. Save every tree as 5 simple lists
trees = []
for tree in web_rf.estimators_:                          # go through the 30 tre
    trees.append({
        "feature": tree.tree_.feature.tolist(),          # which column each que
        "threshold": tree.tree_.threshold.round(3).tolist(),  # the number it co
        "left": tree.tree_.children_left.tolist(),       # where to go if the an
        "right": tree.tree_.children_right.tolist(),     # where to go if the an
        "value": tree.tree_.value[:, 0, 0].round(1).tolist(),  # the rent at eac
    })

print("Saved", len(trees), "trees")
```

```
Saved 30 trees
```

In [52]:
```
# 2. For each area: its region, median rent and number of listings
areas = {}
for loc in sorted(df["location"].unique()):              # go through every area
    area_df = df[df["location"] == loc]                  # only the listings in
    areas[loc] = {
        "region": area_df["region"].mode()[0],           # the most common regio
        "median": float(area_df["monthly_rent"].median()),
        "count": len(area_df),
    }

# 3. Numbers for the "Market overview" page
market = {
    "total_listings": len(df),
    "median_rent": float(df["monthly_rent"].median()),
    "by_region": df.groupby("region")["monthly_rent"].median().to_dict(),
    "by_type": df.groupby("property_type")["monthly_rent"].median().to_dict(),
    "by_furnished": df.groupby("furnished")["monthly_rent"].median().to_dict(),
}

print("Saved", len(areas), "areas")
```

```
Saved 125 areas
```

In [53]:
```
# 4. Put everything in one dictionary, and save it as model.json
web_model = {
    "columns": list(X.columns),                          # the 137 columns, in t
    "areas": areas,
    "property_types": sorted(df["property_type"].unique().tolist()),
    "furnished": sorted(df["furnished"].unique().tolist()),
    "mae": round(web_mae),
    "market": market,
    "trees": trees,
}

with open("model.json", "w") as f:                       # open a new file for w
    json.dump(web_model, f)                              # write the dictionary

print("Saved model.json - copy it into the React folder: frontend/public/")
```

```
Saved model.json - copy it into the React folder: frontend/public/
```

Step 11: Conclusion

Findings

- Rent is driven mostly by size, furnishing, location and property type.
- Random Forest was the best model: MAE about RM 217 and R^2 0.78.
- For Expensive vs Affordable, Random Forest reached 87% accuracy and AUC 0.94.

Was the objective achieved? Yes. The model predicts rent with reasonable accuracy. The Houses Rental web app uses it to predict the rent of a house, tell tenants if a listing is fairly priced, and show which areas fit their budget.

Limitations

- The data is from one website at one point in time.
- These are asking prices, not the final agreed rents.

- Facilities and completion year were not used.
- The model is less accurate for luxury units, because there are few examples.
- The small model used by the web app is a little less accurate (MAE about RM 240), and in a few areas with little data it can give odd results (e.g. fully furnished slightly cheaper than partially furnished).

In [ ]: